TecniHogar
Laboratorio 3 - Herencia

CC2008 - Programación Orientada a Objetos

Universidad del Valle de Guatemala
Ciclo 2, 2026

Solución en Java, análisis, diseño UML y evidencia de pruebas

La aplicación administra cuatro categorías de servicio mediante herencia y polimorfismo, con reglas de programación, visitas y cobro protegidas por validaciones de estado.

Contenido

1. Requisitos, datos y condiciones de validez.
2. Decisiones por categoría y transiciones de estado.
3. Responsabilidades, constructores y extensibilidad.
4. Diagramas UML incorporados y catálogo de miembros.
5. Pruebas con resultados esperados y obtenidos.

Resultado verificado

Compilación con JDK 26.0.1; código compatible con JDK 17 o posterior. Se ejecutaron 118 comprobaciones automatizadas correctas y una prueba adicional de recuperación del menú ante entradas inválidas.

Los datos se conservan en memoria. Los ejemplos iniciales están pendientes y no generan ingresos.

1. Requisitos funcionales

Información que se conserva durante la ejecución

Entidad	Datos
Orden	Código único, cliente, dirección, horas estimadas decimales, zona, estado y visitas. Puntos en instalaciones; materiales en certificadas; BTU en aire.
Cuadrilla	Código único, capacidad positiva de horas y certificación en refrigeración.
Jornada	Código único, cuadrilla, órdenes asignadas, estado y códigos con resultado registrado.
Ingreso	Código de orden, categoría, concepto (registro o visita adicional) e importe. Es un registro inmutable.

Operaciones

RF01. Cotizar costo y días hábiles sin guardar ni cobrar. RF02. Registrar una orden aceptada, cobrar el importe completo y dejarla pendiente. RF03. Registrar y consultar cuadrillas. RF04. Crear jornadas y asignar o retirar órdenes durante su preparación.

RF05. Iniciar una jornada no vacía, contar una visita por orden y cobrar Q40 cuando sea posterior a la primera. RF06. Registrar un resultado por orden: completada o fallida. RF07. Cerrar automáticamente cuando todas tengan resultado y liberar la cuadrilla. RF08. Consultar órdenes, cuadrillas y jornadas.

RF09. Reportar cantidades por categoría y por cada estado, ingresos separados por concepto, ingreso total y por categoría, y jornadas sin cerrar con horas y capacidad disponible. RF10. Mantener el menú hasta salir; rechazar entradas inválidas sin terminar inesperadamente.

Restricciones generales

Datos solo en memoria; consola; montos con dos decimales. La carga inicial contiene ocho órdenes y tres cuadrillas, sin cobros. Los códigos son únicos dentro de cada entidad, distinguen mayúsculas y se recortan los espacios exteriores.

2. Condiciones de validez

Operación	Precondiciones y efecto si se acepta
Construir orden	Código, cliente y dirección no vacíos; horas > 0; zona válida. Diagnóstico ≤ 1.5h; puntos enteros > 0; materiales > 0; BTU entero de 9000 a 60000; aire solo metropolitano.
Registrar orden	Código de orden no repetido, estado pendiente y cero visitas. El costo se calcula antes de insertar y se cobra una sola vez.
Registrar cuadrilla	Código no vacío ni repetido; capacidad > 0; respuesta de certificación s/n.
Crear jornada	Código no vacío ni repetido; cuadrilla existente sin otra jornada en preparación o en curso.
Asignar	Jornada en preparación; orden pendiente con visitas disponibles; compatible con cuadrilla; suma de horas estimadas ≤ capacidad. No se usan horas facturables.
Retirar	Jornada en preparación y orden perteneciente a ella. Se libera capacidad y la orden vuelve a pendiente. No hay cargos.
Iniciar	Jornada en preparación, no vacía, todas las órdenes asignadas y con visitas disponibles. Se validan todas antes de cambiar cualquiera.
Resultado	Jornada en curso, orden perteneciente a ella y sin resultado previo en esa jornada. Solo puede resolverse una orden en atención.
Entrada	Enteros donde se solicitan; decimales con punto; s/n para decisiones. Un formato inválido cancela la operación en curso y regresa al menú.

Conservación del estado ante rechazo

Las validaciones ocurren antes de las mutaciones. Asignar no cobra; retirar no cobra ni devuelve cobros previos. Iniciar repetidamente, repetir un resultado, exceder capacidad o duplicar un código no agrega ingresos. Las consultas y cotizaciones no alteran colecciones.

Se interpreta que el valor declarado de los materiales debe ser positivo, en concordancia con el rechazo de valores numéricos no positivos pedido por el enunciado. Una visita exitosa en el último intento completa la orden; la cancelación ocurre al fallar el último intento.

3. Decisiones por categoría

Clase responsable	Costo	Días (metro / periferia)	Visitas / cuadrilla
Diagnostico	Q150 / Q225, independiente de horas	1 / 2	3; cualquiera
Instalacion	Base Q100 / Q175 + Q90 × techo(max(horas, puntos × 0.75))	3 / 5	3; cualquiera
InstalacionCertificada	super.costo() + Q120 + 5% de materiales	super.diasAtencion() + 2: 5 / 7	2; cualquiera
MantenimientoAire	Q100 + Q110 × horas estimadas + Q80. No redondea horas.	2 / no permitido	1; certificada en refrigeración

Ubicación de cada decisión

Los constructores concretos validan los datos particulares: máximo de horas, puntos, materiales, BTU y zona permitida. Orden valida los datos comunes. Zona conserva una sola definición de la tarifa base compartida por instalaciones y aire.

Los métodos costo(), diasAtencion() y limiteVisitas() son abstractos en Orden y se implementan en cada categoría. compatible(Cuadrilla) permite cualquier cuadrilla por defecto; MantenimientoAire lo sobrescribe para exigir refrigeración.

Jornada pregunta o.compatible(cuadrilla) y usa o.horas(), sin inspeccionar tipos concretos. Orden consulta su propio limiteVisitas() al resolver una visita fallida. Sistema usa categoria() para agrupar reportes e ingresos; no mantiene listas separadas de categorías ni repite sus reglas.

Precisión monetaria

BigDecimal evita errores de representación binaria. Solo las horas facturables de instalación se redondean hacia arriba; los importes se presentan con dos decimales y HALF_UP. La certificada reutiliza el costo base y añade los conceptos propios.

4. Estados y transiciones

Orden

Origen	Operación / condición	Destino
No registrada	Registro aceptado; también carga inicial sin cobro	PENDIENTE
PENDIENTE	Asignar a jornada válida	ASIGNADA
ASIGNADA	Retirar durante preparación	PENDIENTE
ASIGNADA	Iniciar jornada: visitas + 1; Q40 si visitas > 1	EN_ATENCION
EN_ATENCION	Resultado completado	COMPLETADA
EN_ATENCION	Resultado fallido y visitas < límite	PENDIENTE
EN_ATENCION	Resultado fallido y visitas = límite	CANCELADA

COMPLETADA y CANCELADA son terminales. ASIGNADA evita que una orden forme parte de dos jornadas activas. Una jornada conserva sus órdenes históricas aunque una orden fallida se reprograme; el conjunto resultados distingue las visitas ya resueltas en cada jornada.

Jornada

Origen	Operación / condición	Destino
No creada	Crear para una cuadrilla libre	PREPARACION
PREPARACION	Asignar / retirar válidamente	PREPARACION
PREPARACION	Iniciar con una o más órdenes	EN_CURSO
EN_CURSO	Registrar resultado mientras faltan otras órdenes	EN_CURSO
EN_CURSO	Registrar último resultado	CERRADA

Una jornada cerrada no admite cambios. La cuadrilla queda disponible cuando no tiene ninguna jornada sin cerrar. La disponibilidad se deriva de los estados; no se duplica con un indicador mutable en Cuadrilla.

5. Responsabilidades e inicialización

Clase	Responsabilidad e inicio
Orden (abstracta)	Constructor protegido recibe código, cliente, dirección, horas y zona; valida y conserva datos privados finales. Inicializa PENDIENTE y cero visitas. Protege las transiciones.
Diagnostico / Instalacion / MantenimientoAire	Llaman super(c,n,d,h,z), validan atributos propios y resuelven costo, atención y visitas. Instalacion añade puntos; aire añade BTU.
InstalacionCertificada	Llama super(..., puntos), que a su vez llama a Orden; luego valida y guarda materiales. Reutiliza costo y días de Instalacion.
Cuadrilla	Constructor público valida código y capacidad y recibe la certificación. Datos inmutables.
Jornada	Constructor de paquete recibe código y cuadrilla. Inicializa lista y conjunto vacíos y PREPARACION. Controla asignación, capacidad y resultados.
Sistema	Constructor implícito público inicia mapas y libro vacíos. conDatosIniciales() agrega ocho órdenes sin cobro y tres cuadrillas. Coordina identificadores, cobros y reportes.
Ingreso (record)	Constructor canónico recibe orden, categoría, concepto y monto. Conserva un evento contable inmutable.
Main	Constructor implícito inicializa Scanner y Sistema con semilla. main crea Main y ejecuta el bucle; los métodos de lectura convierten entradas.
Zona / EstadoOrden / EstadoJornada	Enumeraciones para valores válidos. Zona recibe su tarifa base al declarar cada constante.
Validacion	Clase auxiliar de paquete; constructor privado. Métodos estáticos reutilizables para texto, positividad y condiciones.

Las listas que se exponen son copias no modificables. Los métodos que cambian órdenes y jornadas tienen visibilidad de paquete: la consola usa Sistema. Las firmas, tipos, parámetros y visibilidades completos aparecen en el UML y en el catálogo siguiente.

6. Herencia y crecimiento

Por qué la certificada extiende Instalacion

Una instalación eléctrica certificada es una instalación con puntos y con la misma regla de horas y base. La relación permite reutilizar el constructor y horasFacturables(), además de super.costo() y super.diasAtencion(). Sobrescribe categoría, costo, tiempo y límite de visitas. Las referencias de tipo Orden siguen funcionando sin depender de la clase concreta.

Alternativa: heredar directamente de Orden y componer una política de cálculo de instalación. Facilitaría combinar variantes independientes mediante estrategias, pero requeriría más objetos y contratos para este alcance. Duplicar la fórmula en dos subclases directas de Orden haría más probable que futuras correcciones quedaran inconsistentes. La herencia elegida reduce esa duplicación; su costo es que cambios al contrato de Instalacion también afectan a la certificada.

Agregar paneles solares o cámaras

Se crea una subclase de Orden (o de Instalacion si realmente cumple su contrato), con constructor que valide los nuevos datos, implementaciones de categoria(), costo(), diasAtencion() y limiteVisitas(), y compatible(Cuadrilla) si necesita una restricción adicional.

Main debe incorporar la opción de creación y lectura de los datos nuevos. Se agregan pruebas y, si se desea, ejemplos iniciales. El switch de Main únicamente es una fábrica de objetos de consola: no calcula precios ni límites.

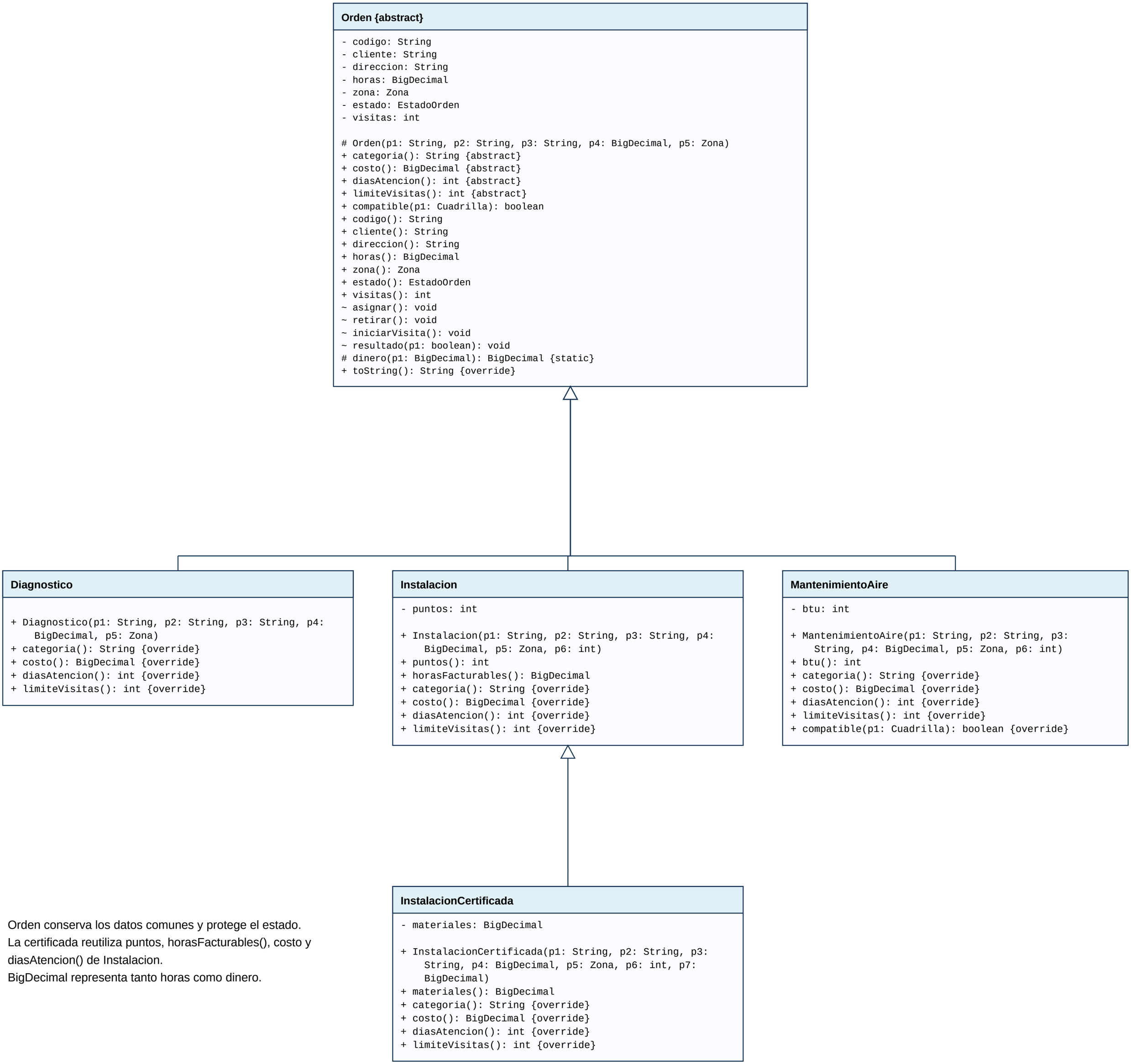
Jornada y los reportes de Sistema no cambian: usan los métodos polimórficos y agrupan por la categoría que cada objeto informa. Si la nueva especialidad requiere una credencial que Cuadrilla todavía no representa, sí será necesario ampliar sus datos y su captura. El flujo de jornadas sigue siendo el mismo.

Contabilidad

Los ingresos son eventos históricos: los ejemplos iniciales no los crean, aunque cada orden conoce su cotización. Un registro aceptado crea un evento REGISTRO; iniciar una visita posterior crea VISITA_ADICIONAL. Los reportes suman eventos reales, sin inferir ingresos a partir de cantidades de órdenes o asignaciones.

7. UML - Jerarquia de servicios

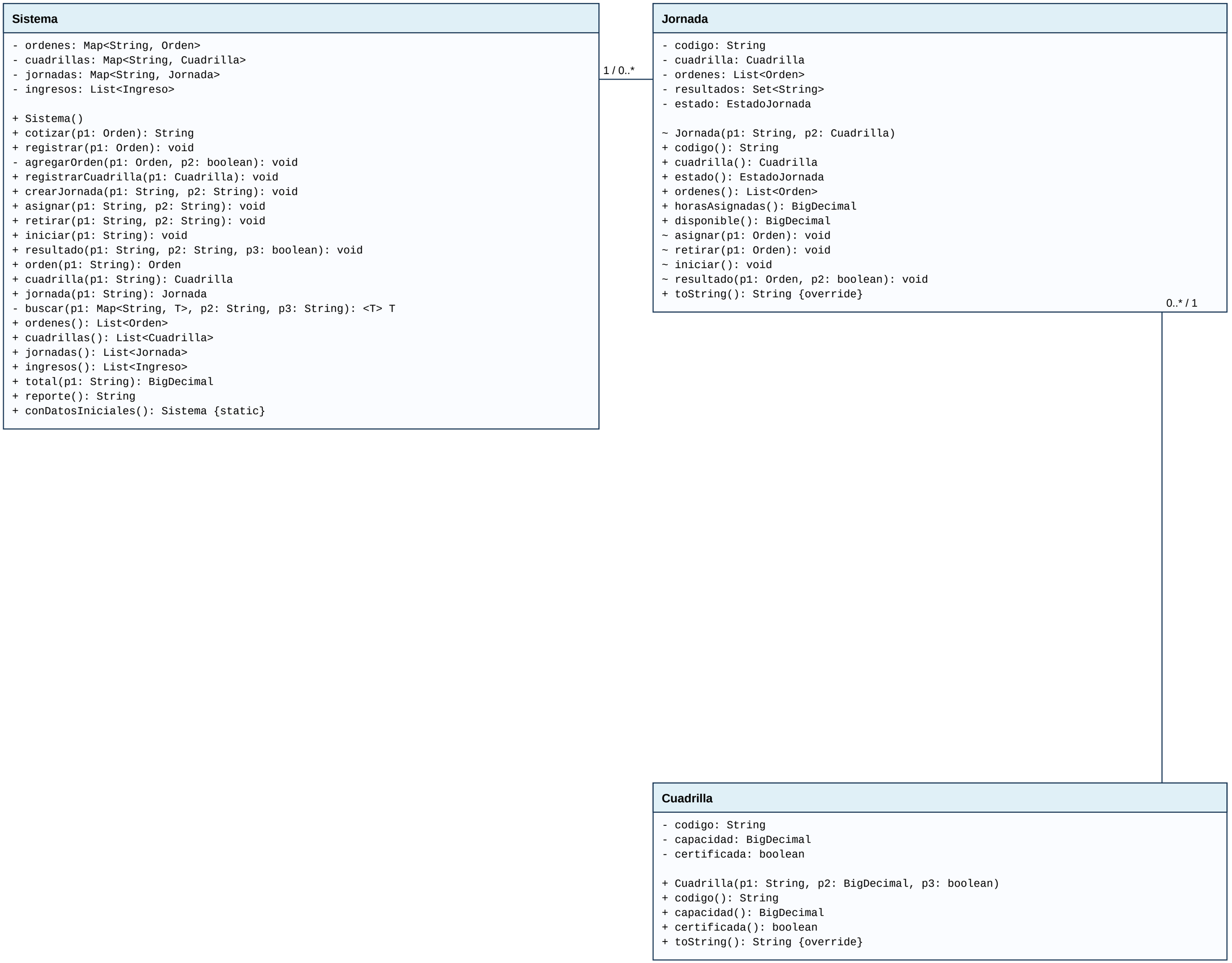
+ publico - privado # protegido ~ paquete {override} sobrescrito {abstract} abstracto



Orden conserva los datos comunes y protege el estado.
La certificada reutiliza puntos, horasFacturables(), costo y diasAtencion() de Instalacion.
BigDecimal representa tanto horas como dinero.

8. UML - Coordinacion y programacion

+ publico - privado # protegido ~ paquete {override} sobrescrito {abstract} abstracto



Sistema contiene 0..* ordenes, cuadrillas, jornadas e ingresos (composicion).
Una jornada conserva 0..* ordenes; una orden puede aparecer en 0..* jornadas historicas. Solo una asignacion activa.
Una cuadrilla puede tener 0..* jornadas historicas y como maximo una sin cerrar.

asignar consulta Orden.compatible(cuadrilla).
iniciar valida todas las ordenes antes de mutar. Sistema registra Q40 por cada visita adicional despues del inicio correcto.
Los reportes usan categoria(), sin comprobar tipos concretos.

9. UML - Consola y tipos auxiliares

+ publico - privado # protegido ~ paquete {override} sobrescrito {abstract} abstracto

Main

```
- entrada: Scanner
- sistema: Sistema

+ Main()
- leer(p1: String): String
- entero(p1: String): int
- decimal(p1: String): BigDecimal
- si(p1: String): boolean
- leerOrden(): Orden
- ejecutar(): void
+ main(p1: String[]): void {static}
```

Validacion

```
- Validacion()
~ texto(p1: String, p2: String): String {static}
~ positivo(p1: BigDecimal, p2: String): BigDecimal {static}
~ exigir(p1: boolean, p2: String): void {static}
```

Zona

```
+ METROPOLITANA: Zona {static}
+ PERIFERICA: Zona {static}
- tarifaBase: BigDecimal

+ values(): Zona[] {static}
+ valueOf(p1: String): Zona {static}
- Zona(p1: String)
+ tarifaBase(): BigDecimal
```

EstadoOrden

```
+ PENDIENTE: EstadoOrden {static}
+ ASIGNADA: EstadoOrden {static}
+ EN_ATENCION: EstadoOrden {static}
+ COMPLETADA: EstadoOrden {static}
+ CANCELADA: EstadoOrden {static}

+ values(): EstadoOrden[] {static}
+ valueOf(p1: String): EstadoOrden {static}
- EstadoOrden()
```

EstadoJornada

```
+ PREPARACION: EstadoJornada {static}
+ EN_CURSO: EstadoJornada {static}
+ CERRADA: EstadoJornada {static}

+ values(): EstadoJornada[] {static}
+ valueOf(p1: String): EstadoJornada {static}
- EstadoJornada()
```

Ingreso

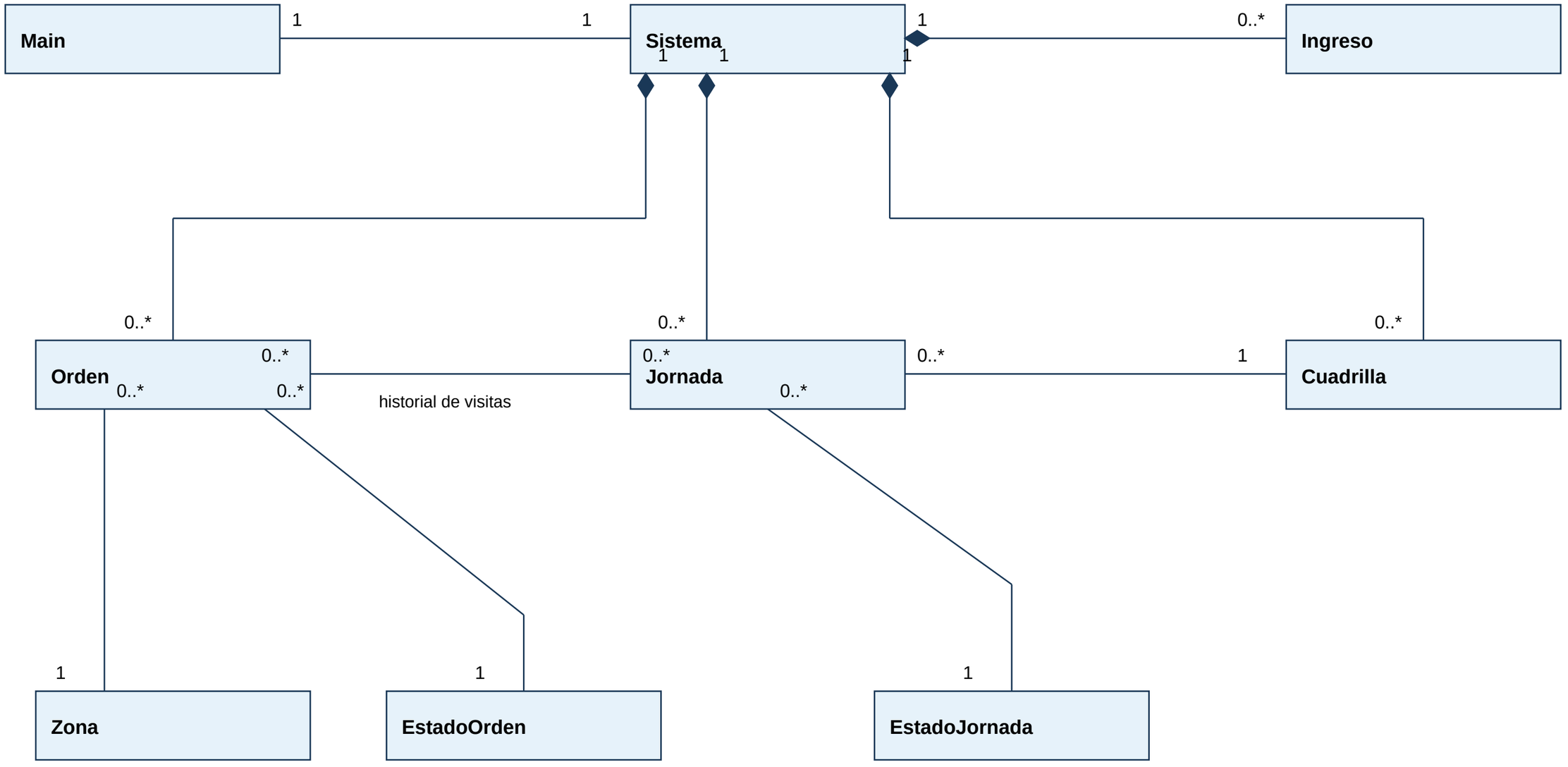
```
- orden: String
- categoria: String
- concepto: String
- monto: BigDecimal

+ Ingreso(p1: String, p2: String, p3: String, p4: BigDecimal)
+ toString(): String
+ hashCode(): int
+ equals(p1: Object): boolean
+ orden(): String
+ categoria(): String
+ concepto(): String
+ monto(): BigDecimal
```

Main contiene main(String[]): void y posee 1 Sistema.
Orden referencia 1 Zona y 1 EstadoOrden.
Jornada referencia 1 EstadoJornada.
Ingreso es un record inmutable; equals, hashCode y toString son generados por Java.
Validacion es una dependencia compartida.

10. UML - Relaciones y multiplicidades

+ publico - privado # protegido ~ paquete {override} sobrescrito {abstract} abstracto



El rombo negro representa composicion. Las entidades pertenecen a un Sistema.
Main, Sistema, Orden, Cuadrilla y Jornada dependen de Validacion (uso de sus metodos estaticos).
MantenimientoAire depende de Cuadrilla para consultar la certificacion.
Restricciones: una orden tiene como maximo una asignacion activa; una cuadrilla tiene como maximo una jornada sin cerrar.
La herencia completa y los miembros se muestran en las tres laminas anteriores.

Catálogo: Orden

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- codigo: String	Identificador único de la entidad.
- cliente: String	Nombre del cliente.
- direccion: String	Lugar del servicio.
- horas: BigDecimal	Horas estimadas usadas para capacidad.
- zona: Zona	Zona que determina tarifa y atención.
- estado: EstadoOrden	Estado actual protegido por las transiciones.
- visitas: int	Cantidad de visitas iniciadas.
# Orden(p1: String, p2: String, p3: String, p4: BigDecimal, p5: Zona)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ categoria(): String {abstract}	Nombre estable para mostrar y agrupar servicios.
+ costo(): BigDecimal {abstract}	Calcula el importe de la categoría sin efectos laterales.
+ diasAtencion(): int {abstract}	Devuelve los días hábiles estimados.

Catálogo: Orden (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	Propósito
+ limiteVisitas(): int {abstract}	Devuelve el máximo de intentos.
+ compatible(p1: Cuadrilla): boolean	Indica si la cuadrilla cumple las restricciones del servicio.
+ codigo(): String	Devuelve codigo; no modifica el estado.
+ cliente(): String	Devuelve cliente; no modifica el estado.
+ direccion(): String	Devuelve direccion; no modifica el estado.
+ horas(): BigDecimal	Devuelve horas; no modifica el estado.
+ zona(): Zona	Devuelve zona; no modifica el estado.
+ estado(): EstadoOrden	Devuelve estado; no modifica el estado.
+ visitas(): int	Devuelve visitas; no modifica el estado.
~ asignar(): void	Valida y asocia una orden a la jornada.
~ retirar(): void	Valida y devuelve una orden a pendiente.

Catálogo: Orden (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
~ iniciarVisita(): void	Incrementa visitas y cambia a EN_ATENCION.
~ resultado(p1: boolean): void	Registra éxito o fallo; puede cancelar o cerrar.
# dinero(p1: BigDecimal): BigDecimal {static}	Redondea importe a dos decimales con HALF_UP.
+ toString(): String {override}	Devuelve una representación legible.

Catálogo: Diagnostico

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ Diagnostico(p1: String, p2: String, p3: String, p4: BigDecimal, p5: Zona)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ categoria(): String {override}	Nombre estable para mostrar y agrupar servicios.
+ costo(): BigDecimal {override}	Calcula el importe de la categoría sin efectos laterales.
+ diasAtencion(): int {override}	Devuelve los días hábiles estimados.
+ limiteVisitas(): int {override}	Devuelve el máximo de intentos.

Catálogo: Instalacion

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- puntos: int	Cantidad de puntos a instalar.
+ Instalacion(p1: String, p2: String, p3: String, p4: BigDecimal, p5: Zona, p6: int)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ puntos(): int	Devuelve puntos; no modifica el estado.
+ horasFacturables(): BigDecimal	Calcula techo del máximo entre horas estimadas y por puntos.
+ categoria(): String {override}	Nombre estable para mostrar y agrupar servicios.
+ costo(): BigDecimal {override}	Calcula el importe de la categoría sin efectos laterales.
+ diasAtencion(): int {override}	Devuelve los días hábiles estimados.
+ limiteVisitas(): int {override}	Devuelve el máximo de intentos.

Catálogo: InstalacionCertificada

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- materiales: BigDecimal	Valor declarado para calcular garantía.
+ InstalacionCertificada(p1: String, p2: String, p3: String, p4: BigDecimal, p5: Zona, p6: int, p7: BigDecimal)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ materiales(): BigDecimal	Devuelve materiales; no modifica el estado.
+ categoria(): String {override}	Nombre estable para mostrar y agrupar servicios.
+ costo(): BigDecimal {override}	Calcula el importe de la categoría sin efectos laterales.
+ diasAtencion(): int {override}	Devuelve los días hábiles estimados.
+ limiteVisitas(): int {override}	Devuelve el máximo de intentos.

Catálogo: MantenimientoAire

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- btu: int	Capacidad del equipo.
+ MantenimientoAire(p1: String, p2: String, p3: String, p4: BigDecimal, p5: Zona, p6: int)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ btu(): int	Devuelve btu; no modifica el estado.
+ categoria(): String {override}	Nombre estable para mostrar y agrupar servicios.
+ costo(): BigDecimal {override}	Calcula el importe de la categoría sin efectos laterales.
+ diasAtencion(): int {override}	Devuelve los días hábiles estimados.
+ limiteVisitas(): int {override}	Devuelve el máximo de intentos.
+ compatible(p1: Cuadrilla): boolean {override}	Indica si la cuadrilla cumple las restricciones del servicio.

Catálogo: Sistema

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- ordenes: Map<String, Orden>	Colección de órdenes; historial en Jornada.
- cuadrillas: Map<String, Cuadrilla>	Registro de cuadrillas por código.
- jornadas: Map<String, Jornada>	Registro de jornadas por código.
- ingresos: List<Ingreso>	Eventos contables efectivamente registrados.
+ Sistema()	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ cotizar(p1: Orden): String	Devuelve costo y plazo sin registrar ni cobrar.
+ registrar(p1: Orden): void	Registra una orden nueva y cobra el costo.
- agregarOrden(p1: Orden, p2: boolean): void	Inserta tras validar; cobrar distingue registro de semilla.
+ registrarCuadrilla(p1: Cuadrilla): void	Agrega una cuadrilla de código no repetido.
+ crearJornada(p1: String, p2: String): void	Crea jornada para una cuadrilla sin jornada abierta.
+ asignar(p1: String, p2: String): void	Valida y asocia una orden a la jornada.

Catálogo: Sistema (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ retirar(p1: String, p2: String): void	Valida y devuelve una orden a pendiente.
+ iniciar(p1: String): void	Inicia jornada; Sistema añade los cargos posteriores a la primera visita.
+ resultado(p1: String, p2: String, p3: boolean): void	Registra éxito o fallo; puede cancelar o cerrar.
+ orden(p1: String): Orden	Código de la orden que generó el evento.
+ cuadrilla(p1: String): Cuadrilla	Cuadrilla responsable de la jornada.
+ jornada(p1: String): Jornada	Consulta del atributo sin modificarlo.
- buscar(p1: Map<String, T>, p2: String, p3: String): <T> T	Busca por código o rechaza sin mutar.
+ ordenes(): List<Orden>	Devuelve ordenes; no modifica el estado.
+ cuadrillas(): List<Cuadrilla>	Devuelve cuadrillas; no modifica el estado.
+ jornadas(): List<Jornada>	Devuelve jornadas; no modifica el estado.
+ ingresos(): List<Ingreso>	Devuelve ingresos; no modifica el estado.

Catálogo: Sistema (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ total(p1: String): BigDecimal	Suma ingresos; concepto null incluye todos.
+ reporte(): String	Genera conteos, ingresos y jornadas abiertas.
+ conDatosIniciales(): Sistema {static}	Crea el sistema con semilla y sin ingresos.

Catálogo: Jornada

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	Propósito
- codigo: String	Identificador único de la entidad.
- cuadrilla: Cuadrilla	Cuadrilla responsable de la jornada.
- ordenes: List<Orden>	Colección de órdenes; historial en Jornada.
- resultados: Set<String>	Códigos con resultado en esta jornada.
- estado: EstadoJornada	Estado actual protegido por las transiciones.
~ Jornada(p1: String, p2: Cuadrilla)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ codigo(): String	Devuelve codigo; no modifica el estado.
+ cuadrilla(): Cuadrilla	Devuelve cuadrilla; no modifica el estado.
+ estado(): EstadoJornada	Devuelve estado; no modifica el estado.
+ ordenes(): List<Orden>	Devuelve ordenes; no modifica el estado.
+ horasAsignadas(): BigDecimal	Suma horas estimadas de las órdenes.

Catálogo: Jornada (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ disponible(): BigDecimal	Calcula capacidad menos horas asignadas.
~ asignar(p1: Orden): void	Valida y asocia una orden a la jornada.
~ retirar(p1: Orden): void	Valida y devuelve una orden a pendiente.
~ iniciar(): void	Inicia jornada; Sistema añade los cargos posteriores a la primera visita.
~ resultado(p1: Orden, p2: boolean): void	Registra éxito o fallo; puede cancelar o cerrar.
+ toString(): String {override}	Devuelve una representación legible.

Catálogo: Cuadrilla

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- codigo: String	Identificador único de la entidad.
- capacidad: BigDecimal	Máximo de horas por jornada.
- certificada: boolean	Credencial de refrigeración.
+ Cuadrilla(p1: String, p2: BigDecimal, p3: boolean)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ codigo(): String	Devuelve codigo; no modifica el estado.
+ capacidad(): BigDecimal	Devuelve capacidad; no modifica el estado.
+ certificada(): boolean	Devuelve certificada; no modifica el estado.
+ toString(): String {override}	Devuelve una representación legible.

Catálogo: Ingreso

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- orden: String	Código de la orden que generó el evento.
- categoria: String	Nombre estable para mostrar y agrupar servicios.
- concepto: String	Distingue registro de visita adicional.
- monto: BigDecimal	Importe inmutable del evento.
+ Ingreso(p1: String, p2: String, p3: String, p4: BigDecimal)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ toString(): String	Devuelve una representación legible.
+ hashCode(): int	Calcula hash de los componentes del record.
+ equals(p1: Object): boolean	Compara por los componentes del record.
+ orden(): String	Devuelve orden; no modifica el estado.
+ categoria(): String	Devuelve categoria; no modifica el estado.
+ concepto(): String	Devuelve concepto; no modifica el estado.

Catálogo: Ingreso (continuación)

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ monto(): BigDecimal	Devuelve monto; no modifica el estado.

Catálogo: Main

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	Propósito
- entrada: Scanner	Lector de entrada estándar.
- sistema: Sistema	Coordinador de la aplicación.
+ Main()	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
- leer(p1: String): String	Lee una línea y recorta espacios.
- entero(p1: String): int	Convierte una línea a entero.
- decimal(p1: String): BigDecimal	Convierte una línea a BigDecimal.
- si(p1: String): boolean	Valida y convierte s/n a boolean.
- leerOrden(): Orden	Construye el servicio seleccionado con los datos de consola.
- ejecutar(): void	Mantiene el menú y recupera errores de entrada.
+ main(p1: String[]): void {static}	Punto de entrada estático de la aplicación.

Catálogo: Validacion

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
- Validacion()	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
~ texto(p1: String, p2: String): String {static}	Valida y devuelve texto no vacío.
~ positivo(p1: BigDecimal, p2: String): BigDecimal {static}	Valida y devuelve BigDecimal positivo.
~ exigir(p1: boolean, p2: String): void {static}	Rechaza si no se cumple la condición.

Catálogo: Zona

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ METROPOLITANA: Zona {static}	Constante válida de la enumeración.
+ PERIFERICA: Zona {static}	Constante válida de la enumeración.
- tarifaBase: BigDecimal	Importe base compartido por zona.
+ values(): Zona[] {static}	Devuelve todas las constantes de la enumeración.
+ valueOf(p1: String): Zona {static}	Busca una constante por nombre.
- Zona(p1: String)	Constructor: recibe los parámetros indicados e inicializa según la sección 5.
+ tarifaBase(): BigDecimal	Devuelve tarifaBase; no modifica el estado.

Catálogo: EstadoOrden

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ PENDIENTE: EstadoOrden {static}	Constante válida de la enumeración.
+ ASIGNADA: EstadoOrden {static}	Constante válida de la enumeración.
+ EN_ATENCION: EstadoOrden {static}	Constante válida de la enumeración.
+ COMPLETADA: EstadoOrden {static}	Constante válida de la enumeración.
+ CANCELADA: EstadoOrden {static}	Constante válida de la enumeración.
+ values(): EstadoOrden[] {static}	Devuelve todas las constantes de la enumeración.
+ valueOf(p1: String): EstadoOrden {static}	Busca una constante por nombre.
- EstadoOrden()	Constructor: recibe los parámetros indicados e inicializa según la sección 5.

Catálogo: EstadoJornada

Visibilidad: + pública, - privada, # protegida, ~ paquete. p1, p2, etc. son parámetros posicionales; los tipos y resultados son los del programa compilado.

Atributo / operación y tipos	**Propósito**
+ PREPARACION: EstadoJornada {static}	Constante válida de la enumeración.
+ EN_CURSO: EstadoJornada {static}	Constante válida de la enumeración.
+ CERRADA: EstadoJornada {static}	Constante válida de la enumeración.
+ values(): EstadoJornada[] {static}	Devuelve todas las constantes de la enumeración.
+ valueOf(p1: String): EstadoJornada {static}	Busca una constante por nombre.
- EstadoJornada()	Constructor: recibe los parámetros indicados e inicializa según la sección 5.

Pruebas: 1 a 14

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Diagnóstico metropolitano 1.5h	150	150.00	OK
Diagnóstico periférico 0.5h	225	225.00	OK
Días diagnóstico metropolitano	1	1	OK
Días diagnóstico periférico	2	2	OK
Instalación 2.5h y 5 puntos: techo de 3.75	4	4	OK
Instalación metropolitana 5 puntos	460	460.00	OK
Instalación periférica 2.1h: techo de 3	445	445.00	OK
Días instalación metropolitana	3	3	OK
Días instalación periférica	5	5	OK
Certificada: 460+120+5% de 2000	680	680.00	OK
Certificada periférica: 175+270+120+50	615	615.00	OK
Días certificada metropolitana	5	5	OK
Días certificada periférica	7	7	OK
Aire 1.5h: 100+165+80	345	345.00	OK

Pruebas: 15 a 28

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Días mantenimiento	2	2	OK
Aire máximo BTU válido	60000	60000	OK
Diagnóstico 1.51h	true	true	OK
Horas cero	true	true	OK
Horas negativas	true	true	OK
Código vacío	true	true	OK
Cliente vacío	true	true	OK
Dirección vacía	true	true	OK
Cero puntos	true	true	OK
Materiales cero	true	true	OK
BTU 8999	true	true	OK
BTU 60001	true	true	OK
Aire periférico	true	true	OK
Capacidad cero	true	true	OK

Pruebas: 29 a 42

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Semilla: ocho órdenes	8	8	OK
Semilla: tres cuadrillas	3	3	OK
Semilla: todas pendientes	true	true	OK
Semilla sin ingresos	0	0.00	OK
Cotizar dos veces no registra	8	8	OK
Cotizar dos veces no cobra	0	0.00	OK
Registro cobra monto completo	150	150.00	OK
Registro duplicado	true	true	OK
Duplicado no cambia cantidad	9	9	OK
Duplicado no cambia ingresos	150	150.00	OK
Cuadrilla duplicada	true	true	OK
Una jornada abierta por cuadrilla	true	true	OK
Jornada duplicada	true	true	OK
Jornada sin órdenes no inicia	true	true	OK

Pruebas: 43 a 56

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Aire con cuadrilla sin certificado	true	true	OK
Rechazo conserva orden pendiente	PENDIENTE	PENDIENTE	OK
Rechazo conserva jornada vacía	0	0	OK
Capacidad usa 2.5h estimadas, no 4 facturables	5.5	5.5	OK
Doble asignación	true	true	OK
Retiro restaura pendiente	PENDIENTE	PENDIENTE	OK
Retiro restaura capacidad	8	8	OK
Cotizaciones, retiros y rechazos sin ingresos	150	150.00	OK
Exceso de capacidad 2.5h > 1.5h	true	true	OK
Capacidad exacta aceptada	0	0.0	OK
Resultado antes de iniciar	true	true	OK
No retirar en curso	true	true	OK
No asignar en curso	true	true	OK
No iniciar dos veces	true	true	OK

Pruebas: 57 a 70

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Resultado exitoso completa	COMPLETADA	COMPLETADA	OK
No repetir resultado	true	true	OK
Completada no se reasigna	true	true	OK
Primera visita y rechazos no cobran extra	0	0.00	OK
Resultado parcial mantiene jornada abierta	EN_CURSO	EN_CURSO	OK
Orden ajena no admite resultado	true	true	OK
Último resultado cierra jornada	CERRADA	CERRADA	OK
Reporte contiene ingreso por categoría	true	true	OK
Reporte excluye jornadas cerradas	false	false	OK
Diagnóstico asignar visita 1 no cobra	150.00	150.00	OK
Diagnóstico contador visita 1	1	1	OK
Diagnóstico ingreso al iniciar visita 1	150.00	150.00	OK
Diagnóstico resultado fallido 1	PENDIENTE	PENDIENTE	OK
Diagnóstico jornada cerrada 1	CERRADA	CERRADA	OK

Pruebas: 71 a 84

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Diagnóstico asignar visita 2 no cobra	150.00	150.00	OK
Diagnóstico contador visita 2	2	2	OK
Diagnóstico ingreso al iniciar visita 2	190.00	190.00	OK
Diagnóstico resultado fallido 2	PENDIENTE	PENDIENTE	OK
Diagnóstico jornada cerrada 2	CERRADA	CERRADA	OK
Diagnóstico asignar visita 3 no cobra	190.00	190.00	OK
Diagnóstico contador visita 3	3	3	OK
Diagnóstico ingreso al iniciar visita 3	230.00	230.00	OK
Diagnóstico resultado fallido 3	CANCELADA	CANCELADA	OK
Diagnóstico jornada cerrada 3	CERRADA	CERRADA	OK
Diagnóstico cancelada no se reasigna	true	true	OK
Diagnóstico rechazo sin cargos	80	80.00	OK
Instalación asignar visita 1 no cobra	190.00	190.00	OK
Instalación contador visita 1	1	1	OK

Pruebas: 85 a 98

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Instalación ingreso al iniciar visita 1	190.00	190.00	OK
Instalación resultado fallido 1	PENDIENTE	PENDIENTE	OK
Instalación jornada cerrada 1	CERRADA	CERRADA	OK
Instalación asignar visita 2 no cobra	190.00	190.00	OK
Instalación contador visita 2	2	2	OK
Instalación ingreso al iniciar visita 2	230.00	230.00	OK
Instalación resultado fallido 2	PENDIENTE	PENDIENTE	OK
Instalación jornada cerrada 2	CERRADA	CERRADA	OK
Instalación asignar visita 3 no cobra	230.00	230.00	OK
Instalación contador visita 3	3	3	OK
Instalación ingreso al iniciar visita 3	270.00	270.00	OK
Instalación resultado fallido 3	CANCELADA	CANCELADA	OK
Instalación jornada cerrada 3	CERRADA	CERRADA	OK
Instalación cancelada no se reasigna	true	true	OK

Pruebas: 99 a 112

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Instalación rechazo sin cargos	80	80.00	OK
Instalación eléctrica certificada asignar visita 1 no cobra	315.00	315.00	OK
Instalación eléctrica certificada contador visita 1	1	1	OK
Instalación eléctrica certificada ingreso al iniciar visita 1	315.00	315.00	OK
Instalación eléctrica certificada resultado fallido 1	PENDIENTE	PENDIENTE	OK
Instalación eléctrica certificada jornada cerrada 1	CERRADA	CERRADA	OK
Instalación eléctrica certificada asignar visita 2 no cobra	315.00	315.00	OK
Instalación eléctrica certificada contador visita 2	2	2	OK
Instalación eléctrica certificada ingreso al iniciar visita 2	355.00	355.00	OK
Instalación eléctrica certificada resultado fallido 2	CANCELADA	CANCELADA	OK
Instalación eléctrica certificada jornada cerrada 2	CERRADA	CERRADA	OK
Instalación eléctrica certificada cancelada no se reasigna	true	true	OK
Instalación eléctrica certificada rechazo sin cargos	40	40.00	OK
Mantenimiento de aire acondicionado asignar visita 1 no cobra	290.00	290.00	OK

Pruebas: 113 a 118

Ejecutadas con `java -cp out tecnihogar.Pruebas pruebas.tsv`. Las filas de rechazo esperan true: la excepción de validación debe producirse. Cada caso conserva sus datos de entrada en la descripción.

Caso / entrada	Esperado	Obtenido	Estado
Mantenimiento de aire acondicionado contador visita 1	1	1	OK
Mantenimiento de aire acondicionado ingreso al iniciar visita 1	290.00	290.00	OK
Mantenimiento de aire acondicionado resultado fallido 1	CANCELADA	CANCELADA	OK
Mantenimiento de aire acondicionado jornada cerrada 1	CERRADA	CERRADA	OK
Mantenimiento de aire acondicionado cancelada no se reasigna	true	true	OK
Mantenimiento de aire acondicionado rechazo sin cargos	0	0.00	OK

Ejecución y entrega

Compilación y prueba

```
javac -encoding UTF-8 -d out src/tecnihogar/*.java
java -cp out tecnihogar.Main
java -cp out tecnihogar.Pruebas pruebas.tsv
```

Resultado obtenido: **118 comprobaciones correctas**. La suite cubre ambas zonas permitidas, horas por puntos y fracciones, garantía, tiempos, capacidad, certificación, resultados parciales, cierre, límites de visitas y efectos sobre ingresos.

Prueba de consola

Entrada: `texto`, luego `1` (cotizar), `9` (categoría inválida), `11` (reporte) y `0` (salir).

Esperado: mensaje de formato numérico inválido, mensaje de categoría inválida, reporte de ocho órdenes pendientes sin ingresos y salida normal. Obtenido: el menú permaneció disponible tras ambos errores; el reporte mostró dos órdenes por categoría y Q0.00 en todos los conceptos. La salida terminó normalmente.

Archivos y publicación

El proyecto incluye fuentes Java, README, evidencia TSV y UML editable. El PDF contiene los diagramas completos, distribuidos en cuatro láminas legibles y complementados por el catálogo de miembros.

La entrega consta de este documento y el repositorio con los archivos fuente Java y las instrucciones de compilación y ejecución.